

LDPC Decoding With Degree-Specific Neural Message Weights and RCQ Decoding

Linfang Wang¹, Caleb Terrill, Dariush Divsalar², *Life Fellow, IEEE*, and Richard D. Wesel¹, *Fellow, IEEE*

Abstract—Recently, neural networks have improved MinSum message-passing decoders for low-density parity-check (LDPC) codes by multiplying or adding weights to the messages, where the weights are determined by a neural network. The neural network complexity to determine distinct weights for each edge is high, often limiting the application to relatively short LDPC codes. Furthermore, storing separate weights for every edge and every iteration can be a burden for hardware implementations. To reduce neural network complexity and storage requirements, this paper proposes a family of weight-sharing schemes that use the same weight for edges that have the same check node degree and/or variable node degree. Our simulation results show that node-degree-based weight-sharing can deliver the same performance requiring distinct weights for each node. This paper also combines these degree-specific neural weights with a reconstruction-computation-quantization (RCQ) decoder to produce a weighted RCQ (W-RCQ) decoder. The W-RCQ decoder with node-degree-based weight sharing has a reduced hardware requirement compared with the original RCQ decoder. As an additional contribution, this paper identifies and resolves a gradient explosion issue that can arise when training neural LDPC decoders.

Index Terms—LDPC decoder, neural decoder, low-bitwidth decoding, hardware efficiency, layered decoding, FPGA.

I. INTRODUCTION

LOW-Density Parity-Check (LDPC) codes [2] have been implemented broadly, including in NAND flash systems and wireless communication systems. Message-passing decoders are often used to decode LDPC codes. Typical message-passing decoders utilize belief propagation (BP), MinSum, and its variations. However, message-passing decoders are sub-optimal because of the existence of cycles in the corresponding Tanner graph.

Manuscript received 27 June 2023; revised 23 October 2023 and 7 December 2023; accepted 7 December 2023. Date of publication 15 December 2023; date of current version 18 April 2024. This work was carried out in part at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration, and National Science Foundation (NSF) grants CCF-2008918 and CCF-1911166 at UCLA. An earlier version of this paper was presented in part at IEEE ISTC 2021 [1]. The associate editor coordinating the review of this article and approving it for publication was G. Liva. (*Corresponding author: Linfang Wang.*)

Linfang Wang, Caleb Terrill, and Richard D. Wesel are with the Department of Electrical and Computer Engineering, University of California at Los Angeles, Los Angeles, CA 90095 USA (e-mail: lfwang@ucla.edu; cterrill26@ucla.edu; wesel@ucla.edu).

Dariush Divsalar is with the Jet Propulsion Laboratory, California Institute of Technology, Pasadena, CA 91109 USA (e-mail: Dariush.Divsalar@jpl.nasa.gov).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCOMM.2023.3343387>.

Digital Object Identifier 10.1109/TCOMM.2023.3343387

Recently, numerous works have been focused on enhancing the performance of message-passing decoders with the help of neural networks (NNs) [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], such as neural belief propagation (N-BP) in [3], normalized MinSum (NMS) and neural OMS decoders in [3], [4], and [6]. The neural network is created by unfolding the message passing operations of each decoding iteration [3]. Each decoding iteration is unfolded into two hidden layers, representing a check node processing layer and a variable node processing layer, and each neuron represents a variable-to-check message (V2C) or a check-to-variable (C2V) message. These neural decoders normally assign each C2V message and/or each V2C message a distinct weight in each iteration and hence are impractical for long-blocklength LDPC codes because the number of required parameters is proportional to the number of edges in the Tanner graph of the parity check matrix.

One solution is to share the weights across iterations or edges in the Tanner graph, like in [5], [13], [15], and [16]. However, these simple weight-sharing methods sacrifice decoding performance in different ways. Besides, the precursor works of literature mainly focus on the short-blocklength codes ($n < 2000$), which may have resulted from the fact that the required memory for training neural decoders with long block lengths by using popular deep learning research platforms, such as PyTorch, exceeds the computation resources that researchers can access. However, as shown in [1] and [13], it is possible to train neural decoders by only using CPUs on personal computers for very long-blocklength codes if resources are handled more efficiently.

On the other perspective, decoders for LDPC codes with low message bit widths are desired when considering the limited hardware resources. Recently, the non-uniformly quantized decoders [18], [19], [20], [21], [22], [23], [24], [25], [26], [27] have shown to deliver excellent performance with very low message precision. One promising decoding paradigm is called reconstruction-computation-quantization (RCQ) decoder [25], [26], [27]. The node operation in an RCQ decoder involves a reconstruction function that allows high-precision message computation and a quantization function that allows low-precision message passing between nodes. Specifically, the reconstruction function, equivalent to a dequantizer, maps the low-bitwidth messages received by a node to high-bitwidth messages for computation. The quantization function quantizes the calculated high-bitwidth messages to low-bitwidth messages that will be sent to its neighbor nodes.

The excellent decoding performance of RCQ decoder comes from its dynamic quantizers and dequantizers that are updated in each layer and each iteration. However, such dynamic quantizers/dequantizers are also overheads of the RCQ decoder in hardware implementation, which may even offset the benefit brought by the low bit-width messages [27].

A. Contribution

This paper proposes a family of weight-sharing schemes for the neural MinSum decoder based on the check node degree and variable node degree. Our simulation results show that the decoders with the node-degree-based weight-sharing schemes can deliver the same performance as the neural MinSum decoder that doesn't share the weights. This paper also combines neural decoding with the RCQ decoding paradigm and proposes a weighted RCQ (W-RCQ) decoder. The W-RCQ decoder with node-degree-based weight sharing has a reduced hardware requirement compared with the RCQ decoder. The contributions of this paper are summarized below:

- *Posterior Joint Training Method.* This paper identifies the gradient explosion issue when training neural LDPC decoders. A posterior joint training method is proposed in this paper to address the gradient explosion problem. Simulation results show posterior joint training delivers better decoding performance than the simple gradient clipping method.
- *Node-Degree-Based Weight Sharing.* This paper illustrates that the weight values of the N-NMS decoder are strongly related to check node degree, variable node degree, and iteration index. As a result, this paper proposes node-degree-based weight-sharing schemes that assign the same weight to the edges with the same check node degree and/or variable node degree.
- *Neural-2D-MinSum decoder.* By employing the node-degree-based weight-sharing schemes on the N-NMS and N-OMS decoders, this paper proposes the N-2D-NMS decoder and N-2D-OMS decoder. 2D means 2-dimensional and implies that the weights in each iteration are shared across two dimensions, i.e., check node degree and variable node degree.
- *W-RCQ Decoder.* This paper applies N-2D-NMS and N-2D-OMS to the RCQ decoding paradigm to introduce a weighted-RCQ (W-RCQ) decoding paradigm. Simulation results for a (9472,8192) LDPC code on a field-programmable gate array (FPGA) device show that compared with the 4-bit RCQ decoder and the 5-bit OMS decoder, the 4-bit W-RCQ decoder delivers comparable FER performance but with reduced hardware requirements.

B. Organization

The remainder of this paper is organized as follows: Section II derives the gradients for a flooding-scheduled N-NMS decoder and shows that the memory to calculate the gradients can be reduced by storing the forward messages compactly. This section also describes the posterior joint training method that addresses the gradient explosion

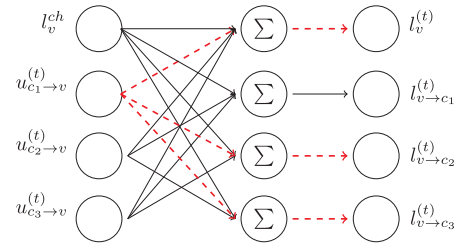


Fig. 1. Computation of V2C messages of a degree-3 variable node v . The red dashed paths show that the gradient of the node $u_{c_i \rightarrow v_j}^{(t)}$ comes from the nodes $l_v^{(t)}$, $l_{v \rightarrow c_2}^{(t)}$ and $l_{v \rightarrow c_3}^{(t)}$.

issue. Section III proposes node-degree-based weight-sharing schemes for neural MinSum decoder, which leads to a family of neural-2D-MinSum decoders. Section IV gives the W-RCQ decoding structure and describes how to train W-RCQ parameters via a quantized neural network (QNN). The simulation results are presented in Section V, and Section VI concludes our work.

II. TRAINING NEURAL MINSUM DECODERS FOR LONG BLOCKLENGTH CODES

For the neural network corresponding to a neural LDPC decoder, the number of neurons in each hidden layer equals the number of edges in the Tanner graph corresponding to the parity check matrix [3]. For the popular NN platforms, such as PyTorch, each neuron requires a data structure that stores the value of the neuron, the gradient of the neuron, the connection of this neuron with other neurons, and so on. Therefore, training neural decoders for long-blocklength LDPC codes in PyTorch demands significant memory, which poses a challenge for researchers with limited resources.

However, the data structure used in PyTorch is redundant to the neural LDPC decoders. One reason is that the neuron connections between hidden layers are repetitive and can be interpreted by the parity check matrix. This immediately reduces the required memory. This section uses N-NMS decoder to show that the memory required to calculate gradients of the neural MinSum decoders can be further reduced by compactly storing the messages in forward propagation.

A. Forward Propagation of N-NMS Decoder

Let $H \in \mathbb{F}_2^{(n-k) \times n}$ be the parity check matrix of an (n, k) binary LDPC code, where n is the codeword length and k is dataword length. Denote i^{th} variable node and j^{th} check node by v_i and c_j , respectively. Let $\text{sgn}(\cdot)$ be the sign function, i.e., $\text{sgn}(x) = 1$ when $x \geq 0$ and $\text{sgn}(x) = -1$ when $x < 0$. For the flooding-scheduled decoder, in the t^{th} decoding iteration, N-NMS decoder updates the C2V message, $u_{c_j \rightarrow v_i}^{(t)}$, by:

$$u_{c_i \rightarrow v_j}^{(t)} = \beta_{(c_i, v_j)}^{(t)} \times \prod_{v_{j'} \in \mathcal{N}(c_i) \setminus \{v_j\}} \text{sgn}(l_{v_{j'} \rightarrow c_i}^{(t-1)}) \times \min_{v_{j'} \in \mathcal{N}(c_i) \setminus \{v_j\}} |l_{v_{j'} \rightarrow c_i}^{(t-1)}|, \quad (1)$$

$\mathcal{N}(c_i)$ is the set of variable nodes that connect c_i and $\{\beta_{(c_i, v_j)}^{(t)} | i \in \{1, \dots, (n-k)\}, j \in \{1, \dots, n\}, H(i, j) =$

$1, t \in \{1, \dots, I_T\}$ is the set of trainable parameters. I_T represents the maximum iterations. The V2C message, $l_{v_i \rightarrow c_j}^{(t)}$, and posterior of each variable node, $l_{v_i}^{(t)}$, of N-NMS decoder in iteration t are calculated by:

$$l_{v_j \rightarrow c_i}^{(t)} = l_{v_j}^{ch} + \sum_{c_{i'} \in \mathcal{N}(v_j) \setminus \{c_i\}} u_{c_{i'} \rightarrow v_j}^{(t)}, \quad (2)$$

$$l_{v_j}^{(t)} = l_{v_j}^{ch} + \sum_{c_{i'} \in \mathcal{N}(v_j)} u_{c_{i'} \rightarrow v_j}^{(t)}. \quad (3)$$

$\mathcal{N}(v_j)$ represents the set of the check nodes connected to v_j . $l_{v_j}^{ch}$ is the log-likelihood ratio (LLR) of the channel observation of v_j . The decoding stops when all parity check nodes are satisfied or I_T is reached.

B. Backward Propagation of N-NMS

Let J be some loss function for the N-NMS neural network, for example, the multi-loss cross entropy in [3]. Denote the gradients of loss J with respect to (w.r.t.) the trainable weights, the C2V message and V2C message by $\frac{\partial J}{\partial \beta_{(c_i, v_j)}^{(t)}}$, $\frac{\partial J}{\partial u_{c_i \rightarrow v_j}^{(t)}}$, and

$\frac{\partial J}{\partial l_{v_j \rightarrow c_i}^{(t)}}$ respectively.

Fig. 1 shows the calculation of the V2C messages for a degree-3 variable node v that connects check nodes c_1 , c_2 , and c_3 . The gradients of these V2C messages are components of the gradients of the C2V messages. As shown by the red dashed paths, $u_{c_1 \rightarrow v}^{(t)}$ is used to calculate $l_{v \rightarrow c_2}^{(t)}$, $l_{v \rightarrow c_3}^{(t)}$, and $l_v^{(t)}$, therefore the gradient $\frac{\partial J}{\partial u_{c_1 \rightarrow v}^{(t)}}$ is accumulated by these three terms. Generally, in iteration t , $\frac{\partial J}{\partial u_{c_i \rightarrow v_j}^{(t)}}$ is updated by:

$$\frac{\partial J}{\partial u_{c_i \rightarrow v_j}^{(t)}} = \frac{\partial J}{\partial l_{v_j}^{(t)}} + \sum_{c_{i'} \in \mathcal{N}(v_j) \setminus \{c_i\}} \frac{\partial J}{\partial l_{v_j \rightarrow c_{i'}}^{(t)}}. \quad (4)$$

In iteration t , when calculating C2V messages, check node c_i receives minimum and second minimum magnitude values, denoted by $\min 1_{c_i}^t$ and $\min 2_{c_i}^t$ from variable nodes $\text{pos} 1_{c_i}^t$ and $\text{pos} 2_{c_i}^t$, respectively, where

$$\begin{aligned} \min 1_{c_i}^t &= \min_{v_{j'} \in \mathcal{N}(c_i)} |l_{v_{j'} \rightarrow c_i}^{(t-1)}|, \\ \text{pos} 1_{c_i}^t &= \text{argmin}_{v_{j'} \in \mathcal{N}(c_i)} |l_{v_{j'} \rightarrow c_i}^{(t-1)}|. \end{aligned} \quad (5)$$

$$\begin{aligned} \min 2_{c_i}^t &= \min_{v_{j'} \in \mathcal{N}(c_i) / \{\text{pos} 1_{c_i}^t\}} |l_{v_{j'} \rightarrow c_i}^{(t-1)}|, \\ \text{pos} 2_{c_i}^t &= \text{argmin}_{v_{j'} \in \mathcal{N}(c_i) / \{\text{pos} 1_{c_i}^t\}} |l_{v_{j'} \rightarrow c_i}^{(t-1)}|. \end{aligned} \quad (6)$$

Only $\min 1_{c_i}^t$ and $\min 2_{c_i}^t$ are used for C2V messages calculation. Fig. 2 illustrates an example of computing the C2V messages of a degree-3 check node c that connects variable nodes v_1 , v_2 , and v_3 . Fig. 2 assumes $\text{pos} 1_c^{(t)} = v_3$ and $\text{pos} 2_c^{(t)} = v_1$.

It can be seen from Fig. 2 that $\frac{\partial J}{\partial \beta_{(c_i, v_j)}^{(t)}}$ is calculated using $\frac{\partial J}{\partial u_{c_i \rightarrow v_j}^{(t)}}$ by:

$$\frac{\partial J}{\partial \beta_{(c_i, v_j)}^{(t)}} = u_{c_i \rightarrow v_j}^{(t)*} \frac{\partial J}{\partial u_{c_i \rightarrow v_j}^{(t)}}, \quad (7)$$

where $u_{c_i \rightarrow v_j}^{(t)*} = \frac{u_{c_i \rightarrow v_j}^{(t)}}{\beta_{(c_i, v_j)}^{(t)}}$. $u_{c_i \rightarrow v_j}^{(t)*}$ is the output of check node Min operation and hence can be calculated efficiently by $\text{sgn}(l_{v_j \rightarrow c_i}^{(t-1)})$, $\min 1_{c_i}^t$, $\min 2_{c_i}^t$, $\text{pos} 1_{c_i}^t$.

In Fig. 2, $l_{v_3 \rightarrow c}^{(t-1)}$ is used to compute $u_{c \rightarrow v_1}^{(t)}$ and $u_{c \rightarrow v_2}^{(t)}$, $l_{v_1 \rightarrow c}^{(t-1)}$ is used to compute $u_{c \rightarrow v_3}^{(t)}$. $l_{v_2 \rightarrow c}^{(t-1)}$ is not involved in computing any C2V messages. As a result, only $l_{v_1 \rightarrow c}^{(t-1)}$ and $l_{v_3 \rightarrow c}^{(t-1)}$ accumulate the gradients, as shown in the blue dashed and red dash-dotted paths in Fig. 2, respectively. Generally, for all variable nodes connected to the check node c_i , only $l_{c_i \rightarrow \text{pos} 1_{c_i}^{(t)}}^{(t-1)}$ and $l_{c_i \rightarrow \text{pos} 2_{c_i}^{(t)}}^{(t-1)}$ receive backward information. Note that the sign operation makes gradient 0, and min operation passes the gradient to the neuron that provides the minimum value. Hence, $\frac{\partial J}{\partial l_{v_j \rightarrow c_i}^{(t-1)}}$ is computed as follows:

$$\begin{cases} \text{sgn}(l_{v_j \rightarrow c_i}^{(t-1)}) \sum_{v_{j'} \in \mathcal{N}(c_i) \setminus \{v_j\}} \frac{\partial J}{\partial |u_{c_i \rightarrow v_{j'}}^{(t)*}|}, & v_j = \text{pos} 1_{c_i}^{(t)} \\ \text{sgn}(l_{v_j \rightarrow c_i}^{(t-1)}) \frac{\partial J}{\partial |u_{c_i \rightarrow \text{pos} 1_{c_i}^{(t)}}^{(t)*}|}, & v_j = \text{pos} 2_{c_i}^{(t)} \\ 0, & \text{otherwise.} \end{cases} \quad (8)$$

The term $\frac{\partial J}{\partial |u_{c_i \rightarrow v_j}^{(t)*}|}$ is calculated by:

$$\frac{\partial J}{\partial |u_{c_i \rightarrow v_j}^{(t)*}|} = \text{sgn}(u_{c_i \rightarrow v_j}^{(t)*}) \beta_{(c_i, v_j)}^{(t)} \frac{\partial J}{\partial u_{c_i \rightarrow v_j}^{(t)}}. \quad (9)$$

(4)-(9) indicate that the neuron values in each hidden layer can be stored compactly with $\text{sgn}(l_{v_j \rightarrow c_i}^{(t)})$, $\min 1_{c_i}^t$, $\min 2_{c_i}^t$, $\text{pos} 1_{c_i}^t$ and $\text{pos} 2_{c_i}^t$. The compactly stored neural values in the hidden layers significantly reduce memory.

C. Posterior Joint Training

Eq. (20) implies that in iteration t , for all variable nodes that connect check node c , only $\text{pos} 1_c^t$ and $\text{pos} 2_c^t$ receive gradients from c . Besides, $|\mathcal{N}(c)| - 1$ gradient terms flow to $\text{pos} 1_c^t$. Hence, if check node c has a large degree, the gradient of J w.r.t. $\text{pos} 1_c^t$ can have a large magnitude, and this large-magnitude gradient will be propagated to the neurons in the preceding layer corresponded to the C2V messages whose check nodes (other than c) connect $\text{pos} 1_c^t$. As a result, the large-magnitude gradients are accumulated and propagated as back propagation proceeds, which results in gradient explosion.

Fig. 3a illustrates the gradient explosion phenomenon when training a flooding-scheduled N-NMS decoder for a (3096,1032) LDPC code. Define $\mu^{(t)}$ as the average magnitude of the gradients of J w.r.t. all C2V messages in iteration t . The gradients are calculated by feeding the NN corresponding to N-NMS decoder with a random input sample and then performing backward propagation. Fig. 3a plots $\mu^{(t)}$ in each decoding iteration. The maximum check node degree and variable node degree of the code are 19 and 27, respectively. The maximum number of decoding iterations of the decoder is 50. It can be seen that the $\mu^{(t)}$ increases exponentially with the decrease of decoding iteration t .

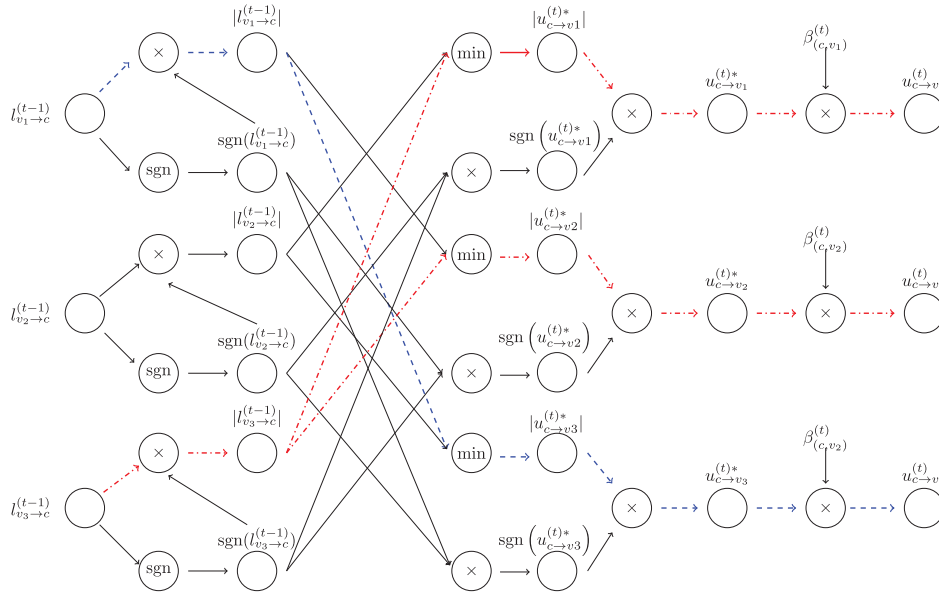


Fig. 2. Computation of V2C messages of a degree-3 check node c . The example assumes that v_3 and v_1 provide the first and second minimum values, respectively. As a result, only $l_{v_1 \rightarrow c}^{(t-1)}$ and $l_{v_3 \rightarrow c}^{(t-1)}$ accumulate the gradients, which are illustrated as the blue dashed and red dash-dotted paths, respectively.

Eq. (7) indicates that large magnitude of $\frac{\partial J}{\partial u_{(c,v)}^{(t)}}$ leads to large magnitude of $\frac{\partial J}{\partial \beta_{(c,v)}^{(t)}}$ and hence prevents the neural network from optimizing weights effectively. To our knowledge, this paper is the first to report the gradient explosion issue for neural LDPC decoder training. However, there have been several techniques that solve the gradient explosion problem:

- 1) *Gradient Clipping*. Gradient explosion is a common problem in the deep learning field, such as recurrent neural networks, and one way to solve this problem is gradient clipping [28]. The gradient clipping in this paper means to limit the maximum gradient magnitude to be some threshold l .
- 2) *Greedy Training*. Dai et al. in [29] proposed greedy training. Greedy training trains the parameters in t^{th} decoding iteration by fixing the pre-trained parameters in the first $t-1$ iterations. Greedy training solves the gradient explosion problem because the large magnitude gradients won't be accumulated and propagated to the preceding hidden layers. However, greedy training requires a time complexity that is proportional to I_T^2 , because one must have trained first $(t-1)$ iterations in order to train the t^{th} decoding iteration.

Eq. (4) indicates that the gradient of J w.r.t. $u_{c_i \rightarrow v_j}^{(t)}$ comes from two parts: the first part is from the posterior $l_{v_j}^{(t)}$, and the second part is from the V2C messages $l_{v_j \rightarrow c_i'}^{(t)}$, $c_i' \in \mathcal{N}(v_j) \setminus \{c_i\}$. Based on the previous analysis, if any $l_{v_j \rightarrow c_i'}^{(t)}$, $c_i' \in \mathcal{N}(v_j) \setminus \{c_i\}$ provides a large magnitude gradient, the neuron $u_{c_i \rightarrow v_j}^{(t)}$ can also have a large magnitude gradient. This will result in a large magnitude to the gradient of J w.r.t. $\beta_{(c_i, v_j)}^{(t)}$, as indicated by (7). In this paper, we propose posterior joint training, which calculates the gradient of J w.r.t. $u_{c_i \rightarrow v_j}^{(t)}$ only using the posterior $l_{v_j}^{(t)}$. More explicitly, for the flooding-

scheduled N-NMS neural network, $\frac{\partial J}{\partial u_{c_i \rightarrow v_j}^{(t)}}$ is calculated by:

$$\frac{\partial J}{\partial u_{c_i \rightarrow v_j}^{(t)}} = \frac{\partial J}{\partial l_{v_j}^{(t)}}. \quad (10)$$

Hence, the gradient of J w.r.t. $\beta_{(c_i, v_j)}^{(t)}$ is calculated as:

$$\frac{\partial J}{\partial \beta_{(c_i, v_j)}^{(t)}} = \frac{u_{c_i \rightarrow v_j}^{(t)}}{\beta_{(c_i, v_j)}^{(t)}} \frac{\partial J}{\partial u_{(c_i, v_j)}^{(t)}} = \frac{u_{c_i \rightarrow v_j}^{(t)}}{\beta_{(c_i, v_j)}^{(t)}} \frac{\partial J}{\partial l_{v_j}^{(t)}}. \quad (11)$$

By calculating the gradients of neurons in the t^{th} decoding iteration only using the posterior $l_{v_j}^{(t)}$, (10) and (11) prevent the large-magnitude gradients from being propagated to the preceding hidden layers. The posterior training equivalently treats each decoding iteration as the last iteration, because $\frac{\partial J}{\partial u_{c_i \rightarrow v_j}^{(t)}}$ in the last iteration is calculated by (10). Besides, (10) is also used in the greedy training [29], because the greedy training trains the parameters of iteration t by assuming the decoder has a maximum iteration of t and fixing the pre-trained parameters of previous $t-1$ iterations. However, the posterior joint training optimizes parameters of all decoding iterations jointly and hence requires a time complexity proportional to I_T .

Unlike the flooding scheduled decoder, which calculates the V2C message using the C2V messages all from the previous iterations, the layered-scheduled decoder facilitates the most recently updated C2V messages to calculate V2C messages. As a result, the $u_{c_i \rightarrow v_j}^{(t)}$ is used to calculate the following terms: 1) Soft decision in iteration t , $l_{v_j}^{(t)}$; 2) V2C messages in iteration t , $l_{v_j \rightarrow c_i'}^{(t)}$, where $i' \in \{i^* | c_i^* \in \mathcal{N}(v_j), i^* > i\}$; and 3) V2C messages in iteration $t+1$, $l_{v_j \rightarrow c_i'}^{(t+1)}$, where $i' \in$

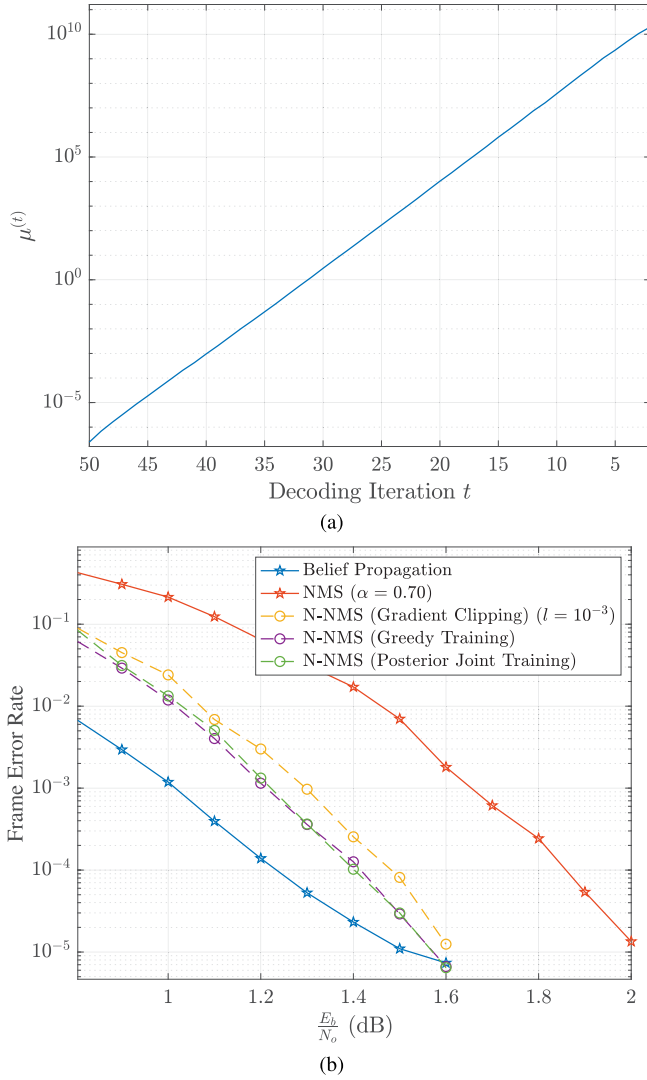


Fig. 3. Fig. (a): The average magnitude of gradients of loss J w.r.t. C2V messages in each decoding iteration. Fig. (b): FER curves of the flooding-scheduled N-NMS decoders for a (3096,1032) LDPC code. Gradient clipping, greedy training, and posterior joint training are used to address the gradient explosion issue.

$\{i^* | c_{i^*} \in \mathcal{N}(v_j), i^* < i\}$. Hence, $\frac{\partial J}{\partial u_{c_{i^*} \rightarrow v_j}^{(t)}}$ is calculated by:

$$\begin{aligned} \frac{\partial J}{\partial u_{c_{i^*} \rightarrow v_j}^{(t)}} &= \frac{\partial J}{\partial l_{v_j}^{(t)}} + \sum_{i' \in \{i^* | c_{i^*} \in \mathcal{N}(v_j), i' > i\}} \frac{\partial J}{\partial l_{v_j \rightarrow c_{i'}}^{(t)}} \\ &+ \sum_{i' \in \{i^* | c_{i^*} \in \mathcal{N}(v_j), i' < i\}} \frac{\partial J}{\partial l_{v_j \rightarrow c_{i'}}^{(t+1)}}. \end{aligned} \quad (12)$$

Posterior joint training abandons the last term in (12) and calculates $\frac{\partial J}{\partial u_{c_{i^*} \rightarrow v_j}^{(t)}}$ as follows:

$$\frac{\partial J}{\partial u_{c_{i^*} \rightarrow v_j}^{(t)}} = \frac{\partial J}{\partial l_{v_j}^{(t)}} + \sum_{i' \in \{i^* | c_{i^*} \in \mathcal{N}(v_j), i^* > i\}} \frac{\partial J}{\partial l_{v_j \rightarrow c_{i'}}^{(t)}}. \quad (13)$$

Fig. 3b shows the frame error rate (FER) of flooding-scheduled N-NMS decoders for a (3096,1032) LDPC code. The maximum decoding iteration time is 50. All three methods to prevent gradient explosion are implemented. The gradient

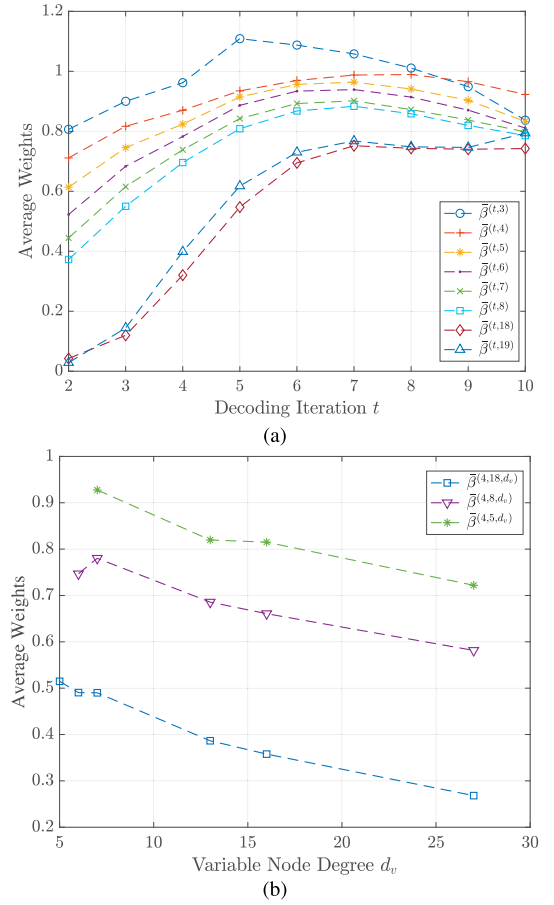


Fig. 4. Mean values of messages of a flooding-scheduled N-NMS decoder for a (3096,1032) LDPC code in each iteration show strong correlations to check node degree and variable node degree.

clipping uses a threshold of $l = 10^{-3}$. The performance of BP and NMS decoders with the same decoding schedule and maximum decoding iteration is also compared. The NMS decoder uses a factor of 0.7. The simulation results show that greedy training and posterior joint training have similar FER curves and perform better than gradient clipping. However, posterior joint training has a lower time complexity than greedy training.

III. NODE-DEGREE-BASED WEIGHT SHARING

N-NMS and N-OMS decoders for the long-blocklength LDPC codes are impractical, because the number of parameters of these decoders is proportional to the number of edges in the corresponding Tanner graph. Weight sharing [30] solves this problem by assigning one weight to different neurons in the NN. Different weight-sharing schemes have been proposed to reduce the number of neural weights in N-NMS and N-OMS decoders. However, simple weight-sharing schemes, such as across iterations or edges in [15] and [16], degrade the decoding performance in different degrees.

This section proposes the node-degree-based weight-sharing schemes that assign the same weights to the edges with the same check node degree and/or variable node degree. We call the N-NMS and N-OMS decoder with node-degree-based weight-sharing schemes by neural 2-dimensional NMS

(N-2D-NMS) decoder and neural 2-dimensional OMS (N-2D-OMS) decoder, respectively, because they are similar to the 2D-MS decoders in [31] and [32]. Simulation results in Section V show that the N-2D-NMS decoder can deliver the same decoding performance with the N-NMS decoder.

A. Motivation

In this subsection, we investigate the relationship between the neural weights of a flooding-scheduled N-NMS decoder and node degrees. The N-NMS decoder is trained for a (3096, 1032) LDPC code, the same one used in Section II-C. The maximum decoding iteration is 10.

Define the set of neural weights of N-NMS decoder that are associated to check node degree d_c in the t^{th} decoding iteration by $\mathcal{B}^{(t,d_c)}$, and $\mathcal{B}^{(t,d_c)} = \{\beta_{(c_i,v_j)}^{(t)} | \deg(c_i) = d_c\}$. Let $\bar{\beta}^{(t,d_c)}$ be the mean value of $\mathcal{B}^{(t,d_c)}$. Fig. 4a shows $\bar{\beta}^{(t,d_c)}$ versus decoding iteration t with all possible check node degrees. The simulation result shows a clear relationship between check node degree and $\bar{\beta}^{(t,d_c)}$, i.e., a larger check node degree corresponds to a smaller $\bar{\beta}^{(t,d_c)}$. This difference is significant in the first few iterations. Additionally, $\bar{\beta}^{(t,d_c)}$ changes drastically in first few iterations for all check node degrees.

In order to explore the relationship between the weights and variable node degrees given a check node degree d_c and decoding iteration index t , we further define $\mathcal{B}^{(t,d_c,d_v)} = \{\beta_{(c_i,v_j)}^{(t)} | \deg(c_i) = d_c, \deg(v_j) = d_v\}$. We denote the average value of $\mathcal{B}^{(t,d_c,d_v)}$ by $\bar{\beta}^{(t,d_c,d_v)}$. Fig. 4b gives the average weights corresponding to various check node degrees and variable node degrees at iteration 4. Statistical results show that, given a specific iteration t and check node degree d_c , a larger d_v indicates a smaller $\bar{\beta}^{(t,d_c,d_v)}$.

In conclusion, the weights of the N-NMS decoder are correlated with the check node degree, the variable node degree, and the decoding iteration index. Thus, node degrees should affect the weighting of messages on their incident edges when decoding LDPC codes. This observation motivates us to propose a family of N-2D-MS decoders.

B. Neural 2D Normalized MinSum Decoders

Based on the previous discussion, it is intuitive to consider assigning the same weights to messages with same check node degree and/or variable node degree. In this subsection, we propose a family of node-degree-based weight-sharing schemes. These weight-sharing schemes can be used on the N-NMS decoder, which gives an N-2D-NMS decoder.

In the t^{th} iteration, a flooding-scheduled N-2D-NMS decoder update $u_{c_i \rightarrow v_j}^{(t)}$ as follows:

$$u_{c_i \rightarrow v_j}^{(t)} = \beta_*^{(t)} \times \prod_{v_{j'} \in \mathcal{N}(c_i) / \{v_j\}} \text{sgn} \left(l_{v_{j'} \rightarrow c_i}^{(t-1)} \right) \times \min_{v_{j'} \in \mathcal{N}(c_i) / \{v_j\}} \left| l_{v_{j'} \rightarrow c_i}^{(t-1)} \right|. \quad (14)$$

$$l_{v_j \rightarrow c_i}^{(t)} = l_{v_i}^{ch} + \alpha_*^{(t)} \sum_{c_{i'} \in \mathcal{N}(v_j) / \{c_i\}} u_{c_{i'} \rightarrow v_j}^{(t)}, \quad (15)$$

TABLE I
VARIOUS NODE-DEGREE-BASED WEIGHT SHARING SCHEMES AND REQUIRED NUMBER OF PARAMETERS PER ITERATION FOR TWO EXAMPLE CODES

Type	$\beta_*^{(t)}$	$\alpha_*^{(t)}$	The number of Required Parameters per Iteration	
			(16200,7200) DVBS-2 code	(3096,1032) PBRL code
No Weight Sharing [3]				
0	$\beta_{(c_i,v_j)}^{(t)}$	1	$4.8 * 10^5$	$1.60 * 10^4$
Weight Sharing Based on Node Degree				
1	$\beta_{(\deg(c_i),\deg(v_j))}^{(t)}$	1	13	41
2	$\beta_{(\deg(c_i))}^{(t)}$	$\alpha_{(\deg(v_j))}^{(t)}$	8	15
3	$\beta_{(\deg(c_i))}^{(t)}$	1	4	8
4	1	$\alpha_{(\deg(v_j))}^{(t)}$	4	7
Weight Sharing Based on Protomatrix				
5 [29]	$\beta_{(\lfloor \frac{i}{f} \rfloor, \lfloor \frac{j}{f} \rfloor)}^{(t)}$	1	—	101
6	$\beta_{(\lfloor \frac{i}{f} \rfloor)}^{(t)}$	1	—	17
7	1	$\alpha_{(\lfloor \frac{j}{f} \rfloor)}^{(t)}$	—	25
Weight sharing based on Iteration [13], [16]				
8	$\beta^{(t)}$	1	1	1

$$l_{v_j}^{(t)} = l_{v_i}^{ch} + \alpha_*^{(t)} \sum_{c_{i'} \in \mathcal{N}(v_j)} u_{c_{i'} \rightarrow v_j}^{(t)}. \quad (16)$$

$\beta_*^{(t)}$ and $\alpha_*^{(t)}$ are the learnable weights. The subscript * is replaced in Table I with the information needed to identify the specific weight depending on the weight-sharing methodology. Table I lists different weight-sharing types, each identified in the first column by a type number. As a special case, we denote type 0 by assigning distinct weights to each edge, i.e., N-NMS decoder. Columns 2 and 3 describe how each type assigns $\beta_*^{(t)}$ and $\alpha_*^{(t)}$, respectively. In this paper, we refer to a decoder that uses a type- x weight-sharing scheme as a type- x decoder.

Types 1-4 assign the same weights based on the node degree. In particular, Type 1 assigns the same weight to the edges that have same check node *and* variable node degree. Type 2 considers the check node degree and variable node degree separately. As a simplification, type 3 and type 4 only consider check node degree and variable node degree, respectively.

Dai et al. in [29] studied weight sharing based on the edge type of multi-edge-type (MET)-LDPC codes, or protograph-based codes. We also consider this metric for types 5, 6, and 7. Type 5 assigns the same weight to the edges with the same edge type, i.e., the edges that belong to the same position in the protomatrix. In Table. I, f is the lifting factor. Types 6 and 7 assign parameters based only on the horizontal (protomatrix row) and vertical layers (protomatrix column), respectively. Finally, type 8 assigns a single weight to all edges in each decoding iteration, as in [13] and [16].

The gradients $\frac{\partial J}{\partial \beta_*^{(t)}}$ and $\frac{\partial J}{\partial \alpha_*^{(t)}}$ in N-2D-NMS decoder are accumulated from the gradients of C2V messages that use $\beta_*^{(t)}$ and $\alpha_*^{(t)}$ in decoding process, respectively. For example, the type-2 N-2D-NMS decoder assigns $\beta_{d_c}^{(t)}$ to all C2V messages with check node degree d_c and assigns $\alpha_{d_v}^{(t)}$ to all C2V

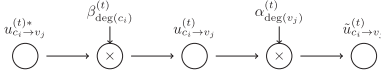


Fig. 5. Relationship between $u_{c_i \rightarrow v_j}^{(t)*}$, $u_{c_i \rightarrow v_j}^{(t)}$, and $\tilde{u}_{c_i \rightarrow v_j}^{(t)}$ in the type-2 N-2D-NMS decoder.

messages with check node degree d_v . As a result,

$$\frac{\partial J}{\partial \beta_{d_c}^{(t)}} = \sum_{(c_i, v_j) \in \mathcal{E}(d_c)} u_{c_i \rightarrow v_j}^{(t)*} \frac{\partial J}{\partial u_{c_i \rightarrow v_j}^{(t)}}, \quad (17)$$

$$\frac{\partial J}{\partial \alpha_{d_v}^{(t)}} = \sum_{(c_i, v_j) \in \mathcal{E}(d_v)} u_{c_i \rightarrow v_j}^{(t)} \frac{\partial J}{\partial \tilde{u}_{c_i \rightarrow v_j}^{(t)}}, \quad (18)$$

where $\mathcal{E}(d_c)$ and $\mathcal{E}(d_v)$ are the set of edges whose check node degree and variable node degree are d_c and d_v , respectively.

$u_{c_i \rightarrow v_j}^{(t)*} = \frac{u_{c_i \rightarrow v_j}^{(t)}}{\beta_{(c_i, v_j)}^{(t)}}$, $\tilde{u}_{c_i \rightarrow v_j}^{(t)} = \alpha_{\deg(v_j)}^{(t)} u_{c_i \rightarrow v_j}^{(t)}$. Fig. 5 gives the relationship between $u_{c_i \rightarrow v_j}^{(t)*}$, $u_{c_i \rightarrow v_j}^{(t)}$, and $\tilde{u}_{c_i \rightarrow v_j}^{(t)}$ in the type-2 N-2D-NMS decoder.

A (3096,1032) LDPC code and the (16200,7200) DVBS-2 [33] standard LDPC code are considered in this section, and the number of parameters per iteration required for various weight-sharing schemes of these two codes are listed in column 4 and 5 in Table. I, respectively. It is shown that the number of parameters required by the node-degree-based weight sharing is less than that required by the protomatrix-based weight sharing.

C. Neural 2D Offset MinSum Decoder

The node-degree-based weight-sharing schemes can be applied to the N-OMS decoder similarly and lead to a neural 2D OMS (N-2D-OMS) decoder. Specifically, a flooding N-2D-OMS decoder updates $u_{c_i \rightarrow v_j}^{(t)}$ by:

$$u_{c_i \rightarrow v_j}^{(t)} = \prod_{v_{j'} \in \mathcal{N}(c_i) \setminus \{v_j\}} \text{sgn}(l_{v_{j'} \rightarrow c_i}^{(t-1)}) \times \text{ReLU} \left(\min_{v_{j'} \in \mathcal{N}(c_i) \setminus \{v_j\}} |l_{v_{j'} \rightarrow c_i}^{(t-1)}| - \beta_*^{(t)} - \alpha_*^{(t)} \right). \quad (19)$$

$\text{ReLU}(x) = \max(0, x)$. The $l_{v_j \rightarrow c_i}^{(t)}$ and $l_{v_j}^{(t)}$ are updated using (2) and (3). For the N-2D-OMS decoders, the constant value 1 in Table I should be replaced by 0.

D. Hybrid Neural Decoder

We consider a hybrid training structure that utilizes a neural network combining feed-forward and recurrent modules to reduce the number of parameters further. The hybrid neural decoder uses distinct neural weights for each of the first I' decoding iterations and uses the same weights for the remaining $I_T - I'$ iterations. For example, for the hybrid neural NMS decoder, the C2V messages are updated by:

$$u_{c_i \rightarrow v_j}^{(t)} = \begin{cases} \beta_{(c_i, v_j)}^{(t)} u_{c_i \rightarrow v_j}^{(t)*}, & t < I' \\ \beta_{(c_i, v_j)}^{(I_T)} u_{c_i \rightarrow v_j}^{(t)*}, & I' \leq t \leq I_T, \end{cases} \quad (20)$$

and the hybrid version of N-2D-NMS decoders can be constructed similarly.

The motivation for the hybrid decoder is from the observation that the neural weights of N-NMS decoder change drastically in the first few iterations but negligibly during the last few iterations, as illustrated in Fig. 4. Therefore, using the same parameters for the last few iterations doesn't cause large performance degradation.

IV. WEIGHTED RCQ DECODER

This section combines the N-2D-NMS or N-2D-OMS decoder with RCQ decoding paradigm and proposes a weighted RCQ (W-RCQ) decoder. Unlike the RCQ decoder, whose quantizers and dequantizers are updated in each iteration (and each layer, if layer-scheduled decoding is considered), the W-RCQ decoder only uses a small number of quantizers and dequantizers during the decoding process. However, the C2V messages of the W-RCQ decoder will be weighted by dynamic node-degree-based parameters that are trained by a QNN.

A. Layered Decoding and RCQ Decoder

The flooding schedule and layered schedule are two decoding schedules widely used in LDPC decoders. As in (1) to (3), the flooding schedule first updates all C2V messages and then all V2C messages in one iteration. The layered schedule, on the other hand, partitions all the check nodes (or variable nodes) to several layers and updates the C2V messages and V2C messages layer by layer. As an example, in the t^{th} iteration, a layered MinSum decoder calculates the messages $u_{c_i \rightarrow v_{j'}}^{(t)}$ and updates the posteriors $l_{v_{j'}}$ as follows:

$$l_{v_{j'}} = l_{v_{j'}} - u_{c_i \rightarrow v_{j'}}^{(t-1)}, \quad \forall v_{j'} \in \mathcal{N}(c_i), \quad (21)$$

$$u_{c_i \rightarrow v_{j'}}^{(t)} = \left(\prod_{v_j \in \mathcal{N}(c_i) \setminus \{v_{j'}\}} \text{sgn}(l_{v_j}^{(t-1)}) \right) \times \min_{v_j \in \mathcal{N}(c_i) \setminus \{v_{j'}\}} |l_{v_j}^{(t-1)}|, \quad \forall v_{j'} \in \mathcal{N}(c_i), \quad (22)$$

$$l_{v_{j'}} = l_{v_{j'}} + u_{c_i \rightarrow v_{j'}}^{(t)}, \quad \forall v_{j'} \in \mathcal{N}(c_i). \quad (23)$$

Low-bit-width decoders with uniform quantizers typically suffer large degradation in decoding performance. The authors in [27] propose the reconstruction-computation-quantization (RCQ) paradigm that facilitates dynamic non-uniform quantization to achieve good decoding performance with low message precision.

Fig. 6a gives a layered MinSum RCQ (msRCQ) decoding diagram. The layered msRCQ decoder follows the equation (21)-(23) but with the non-uniform quantizers $Q^{(t,r)}$ and reconstruction functions $R^{(t,r)}$ designed distinctly for each layer $r = 1, \dots, M$ in iteration $t = 1, \dots, I_T$. M is the number of total layers. The $Q^{(t,r)}$ functions quantize b_v -bit messages to b_c -bit messages, where $b_v > b_c$, and the $R^{(t,r)}$ functions map b_c -bit messages to b_v -bit messages. The dynamic non-uniform $Q^{(t,r)}$ and $R^{(t,r)}$ deliver good decoding performance with a small b_c but also bring extra overhead to store those parameters in hardware. As shown in 6a, extra memory is required to store $Q^{(t,r)}$ and $R^{(t,r)}$ parameters, and extra logic and wires are required to distribute the quantizer

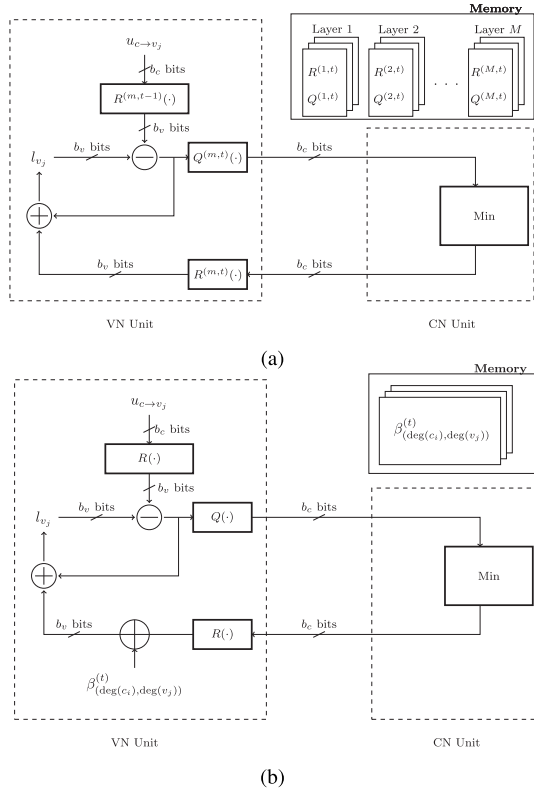


Fig. 6. Decoding diagrams for two distinct versions of layered MinSum decoding: (a) standard RCQ decoder of [27] and (b) proposed weighted (offset) RCQ decoder.

and dequantizers to each variable node (VN) units. As shown in 6a, the overhead to store a larger number of $Q^{(t,r)}$ and $R^{(t,r)}$ functions may offset the benefit brought by decoding with messages of low bit width.

B. Weighted-RCQ Decoder

In this section, we combine the RCQ decoder with the neural decoder to propose a weighted RCQ decoding structure. Fig. 6b gives the decoding paradigm of a layer-scheduled weighted OMS-RCQ decoder (W-OMS-RCQ). One goal of W-RCQ is to reduce memory requirements by reducing the number of quantizer/dequantizer pairs $Q(\cdot)/R(\cdot)$ and instead relying on the scalar parameters $\beta_{(\deg(c_i), \deg(v_j))}^{(t)}$ determined by the neural network to capture most or all of the dynamic adjustments needed to adapt with each iteration. The differences between the W-OMS-RCQ decoder and the msRCQ decoder are summarized as follows:

- *Reconstruction and Quantization.* Unlike the msRCQ decoding diagram in Fig. 6a, the W-OMS-RCQ decoder only uses very few $R(\cdot)$ and $Q(\cdot)$ functions in the decoding, for example, three or fewer, and each $R(\cdot)$ and $Q(\cdot)$ are used for several iterations. This reduces required memory and wire complexity.
- *Message adjustment.* The W-OMS-RCQ decoder weights the reconstructed C2V messages with additive parameters. As shown in Fig. 6b, extra memory is required to store the weights. Besides, the weights in Fig. 6b can be multiplicative, leading to a W-NMS-RCQ decoder.

C. Non-Uniform Quantizer

An important design for a W-RCQ decoder is the quantization and reconstruction (dequantization) function selection. The authors in [27] use discrete density evolution to design dynamic quantizers and dequantizers for the RCQ decoder. For the W-RCQ decoder, this paper considers the quantizers and dequantizers parameterized by the power functions.

Let $Q(x)$ be a symmetric b_c -bit quantizer that features sign information and a magnitude quantizer $Q^*(|x|)$. The magnitude quantizer selects one of 2^{b_c-1} possible indices using the threshold values $\{\tau_0, \tau_1, \dots, \tau_{\max}\}$, where $\tau_j = C \left(\frac{j}{2^{b_c-1}}\right)^\gamma$ for $j \in \{0, \dots, 2^{b_c-1} - 1\}$ and $\tau_{\max}$ is $\tau_{j_{\max}}$ for $j_{\max} = 2^{b_c-1} - 1$. Given an input x , which can be decomposed into sign part $\text{sgn}(x)$ and magnitude part $|x|$, $Q^*(|x|) \in \mathbb{F}_2^{b_c-1}$ is defined by:

$$Q^*(|x|) = \begin{cases} j, & \tau_j \leq |x| < \tau_{j+1} \\ 2^{b_c-1} - 1, & |x| \geq \tau_{\max}, \end{cases} \quad (24)$$

where $0 \leq j \leq j_{\max}$. Let $s(x)$ be the sign bit of x , which is computed by $s(x) = \mathbb{1}(x < 0)$, where $\mathbb{1}(\cdot)$ is the indicator function. Then, $Q(x) = [s(x) \ Q^*(|x|)]$. The thresholds of $Q^*(|x|)$ have a power-function form and are controlled by two parameters. The parameter C defines the maximum magnitude the quantizer can take, and γ manipulates the non-uniformity of the quantizer. Specifically, if $\gamma = 1$, $Q(x)$ becomes a uniform quantizer.

Let $d \in \mathbb{F}_2^{b_c}$ be a b_c -bit message. d can be represented as $[d^{\text{MSB}} \ \tilde{d}]$, where $d^{\text{MSB}} \in \{0, 1\}$ indicates the sign and $\tilde{d} \in \mathbb{F}_2^{b_c-1}$ corresponds to the magnitude. The magnitude reconstruction function $R^*(\tilde{d}) = \tau_{\tilde{d}} = C \left(\frac{\tilde{d}}{2^{b_c-1}}\right)^\gamma$, and $R(d) = (-2d^{\text{MSB}} + 1)R^*(\tilde{d})$. Note that both the magnitude quantization function and magnitude reconstruction function use $\{\tau_0, \dots, \tau_{\max}\}$ as their parameters.

The choice of quantizers is heuristic. We start with one quantizer and tune its C and γ . One-quantizer scheme will be used if the resultant decoder has no error floor above a target FER such as 10^{-6} ; otherwise, we increase the number of quantizers by one each time and tune their parameters until a set of quantizers that don't show error floor is found. Besides, for the case of multiple quantizers, the general rule of tuning parameters is that the quantizer for earlier iterations takes a smaller magnitude.

The other way to optimize the quantizers is to treat the quantizer parameters as the learnable parameters of the neural network and optimize them in the training process, as in [19]. This method will be studied in our future research.

D. Training W-RCQ Decoder via a Quantized Neural Network

Like the neural NMS decoder in [3], the W-RCQ decoder can be unfolded to an NN. The neural network unfolded by the W-RCQ decoder is a QNN because of its quantization and reconstruction functions. The QNN then trains the weights of the W-RCQ decoder. The training was conducted using stochastic gradient descent with mini-batches. Each sample in the mini-batch is a BPSK-modulated all-zero codeword

corrupted by the additive white Gaussian noise whose variance is in the range where a conventional NMS decoder with a factor 0.7 reaches a FER between 10^{-3} and 10^{-2} . In our training experiments, we assign each sample with an $\frac{E_b}{N_0}$ for noise generation. The $\frac{E_b}{N_0}$ of each sample is chosen such that all the $\frac{E_b}{N_0}$ values with a step of 0.1 dB in the specified range are evenly distributed to each mini-batch.

One problem of QNN is that quantization functions result in zeros derivatives almost everywhere. In this work, we use a straight-through estimator (STE) [19], [34] in the backward propagation to solve this issue. The STE uses artificial gradients in QNN training to replace the zero derivative of a quantization function in the chain rule. STE is found to be the most efficient training method for QNNs in [34].

E. Fixed-Point W-RCQ Decoder

This paper uses the pair (b_c, b_v) to denote the bit width for the fixed-point decoders, where b_c is the bit width of C2V messages, and b_v is the bit width of V2C messages and the posteriors of variable nodes. For the W-RCQ decoders, the learnable parameters are first trained under a floating point message representation and then quantized to b_v bits.

V. SIMULATION RESULT AND DISCUSSION

This section evaluates the performance of the N-2D-NMS decoders and the W-RCQ decoders for LDPC codes with different block lengths and code rates. The LDPC codes used in this section are listed in Table II. All the encoded bits are modulated by binary phase-shift keying (BPSK) and transmitted through an Additive White Gaussian Noise (AWGN) channel.

A. (16200,7200) DVBS-2 LDPC Code

Fig. 7a shows the FER performance of N-2D-NMS decoders with various weight sharing types for the (16200, 7200) DVBS-2 LDPC code. The FER performance of BP and NMS decoders is given for comparison. The single multiplicative weight of NMS decoder is 0.88. All decoders are flooding-scheduled, and the maximum decoding iteration is 50. It is shown that the N-NMS decoder (i.e., type-0 decoder) outperforms BP at 1.3 dB with a lower error floor but requires 4.8×10^{-5} parameters in each iteration. The type-1 and type-2 decoders, which share weights based on the check node degree and variable node degree, deliver a slightly better decoding performance than the N-NMS decoder, with only 13 and 8 parameters per iteration, respectively.

Fig. 7a also shows that the FER performance degrades if only considering sharing weights w.r.t. the check node degree (type-3) or the variable node degree (type-4). Type-4 decoder delivers a similar performance to N-NMS decoder; whereas Type-3 decoder is inferior to N-NMS decoder by 0.04 dB. However, both types 3 and 4 require only 4 parameters in each iteration. Fig. 8a and 8b give the $\beta_{(\deg(c_i))}^{(t)}$ and $\alpha_{(\deg(v_j))}^{(t)}$ of type-2 N-2D-NMS decoder, which align with our observation in the previous section; i.e., in each decoding iteration, larger degree node corresponds to a smaller value. Besides, as shown

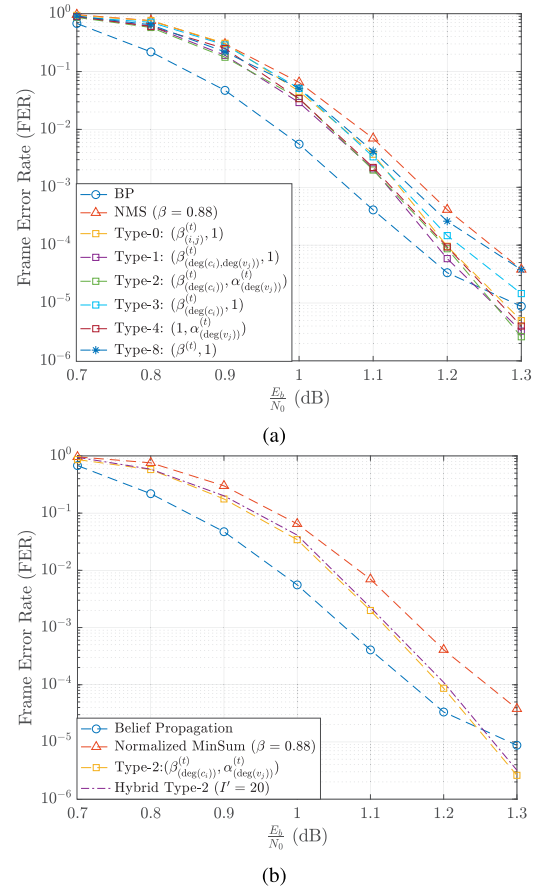


Fig. 7. Fig. (a): The FER performance of the N-2D-NMS decoders with various weight-sharing types for the (16200,7200) DVBS-2 LDPC code. Fig. (b): The FER performance of the hybrid type-2 N-2D-NMS decoder that uses distinct weights in the first 20 iterations and the same weights in the remaining 30 iterations.

in Fig. 8a and 8b, the weights change negligibly after 20th iteration. Thus, the hybrid type-2 N-2D-NMS decoder with $I' = 20$ delivers similar performance to the full feed-forward decoding structure, as shown in Fig. 7b.

B. (9472,8192) Quasi-Cyclic LDPC Code

This subsection designs 3-bit and 4-bit W-OMS-RCQ decoders for a (9472,8192) QC LDPC code and compares them with the fixed-point OMS decoder and RCQ decoders. All decoders in this subsection are layer-scheduled with a maximum iteration of 10. The 4-bit W-OMS-RCQ decoder uses one quantizer/dequantizer pair with $C = 10$, $\gamma = 1.7$ in decoding. The 3-bit W-OMS-RCQ decoder, on the other hand, uses three quantizer/dequantizer pairs. The decoder uses the quantizer/dequantizer with $C = 3$ and $\gamma = 1.3$ in the first six iterations. In iteration 7 to 8, the quantizer/dequantizer has $C = 5$ and $\gamma = 1.3$. The quantizer/dequantizer uses $C = 7$ and $\gamma = 1.3$ in the last two decoding iterations. The pair (b_c, b_v) in the legend gives the bit width of each decoder's check node message and variable node message.

Fig. 9a compares the FER performance of W-OMS-RCQ decoders with msRCQ decoders and a 5-bit OMS decoder. The decoders in Fig. 9a are also implemented using an FPGA device (Xilinx Zynq UltraScale+ MPSoC) for the study of

TABLE II
LDPC CODES USED FOR SIMULATION

Code	Rate	Edge distribution
(16200,7200) DVBS-2 LDPC code [33]	$\frac{4}{9}$	$\lambda(x) = 2.06 * 10^{-5} + 0.3703x + 0.3333x^2 + 0.2963x^7$ $\rho(x) = 0.1186x^3 + 0.3332x^4 + 0.4445x^5 + 0.1037x^6$
(9472,8192) QC-LDPC code [27]	$\frac{8}{9}$	$\lambda(x) = x^3$ $\rho(x) = 0.3919x^{28} + 0.6081x^{29}$
$k = 1032$ PBRL LDPC code [35]	$\frac{8}{9}, \frac{8}{10}, \dots, \frac{8}{24}$	$\lambda(x) = 0.1190 + 0.7940x^4 + 0.0952x^5 + 0.0556x^6 + 0.3095x^{12} + 0.1270x^{16} + 0.2143x^{26}$ $\rho(x) = 0.0238x^2 + 0.0635x^3 + 0.0794x^4 + 0.1905x^5 + 0.2222x^6 + 0.1270x^7 + 0.1429x^{17} + 0.1508x^{18}$

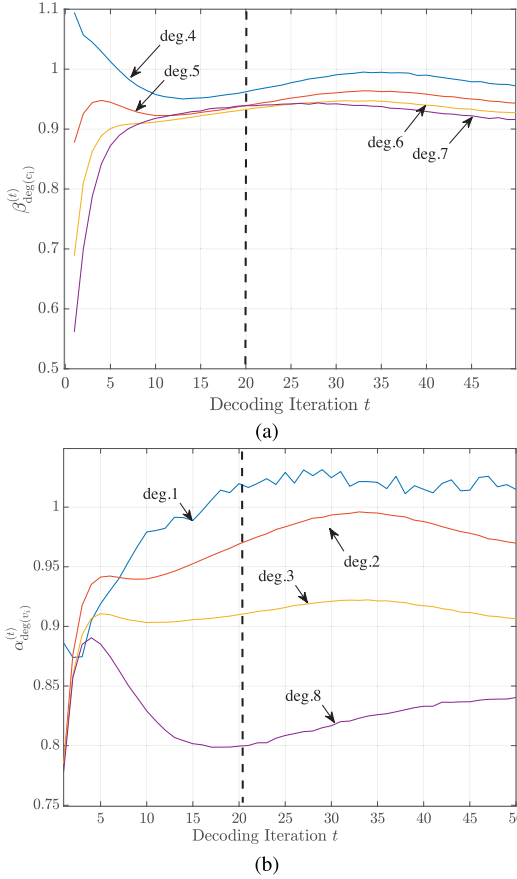


Fig. 8. The change of weights of the type-2 N-2D-NMS decoder for (16200, 7200) DVBS-2 LDPC code w.r.t. check node degree, variable node degree and iteration index. Specifically, Fig. (a) gives $\beta_{(\deg(c_i))}^{(t)}$ for all possible check node degrees in each decoding iteration t , Fig. (b) gives $\alpha_{(\deg(v_j))}^{(t)}$ for all possible variable node degrees in each decoding iteration t .

resource usage. Table III lists the usage of lookup tables (LUTs), registers, block RAM (BRAM), and routed nets of various decoders. For the details of FPGA implementations of the decoders, we refer the readers to [26].

The simulation result shows the 4-bit msRCQ decoder has the best FER performance. The 4-bit W-Oms-RCQ decoder and 5-bit OMS decoder have similar FER performance, which is inferior to the 4-bit msRCQ decoder by 0.01 dB. However, as shown in Table III, the 4-bit W-Oms-RCQ decoder requires much fewer resources than the 4-bit msRCQ decoder and the 5-bit OMS decoder. Compared to the 5-bit OMS decoder, the 3-bit W-Oms-RCQ and 3-bit msRCQ decoder have a

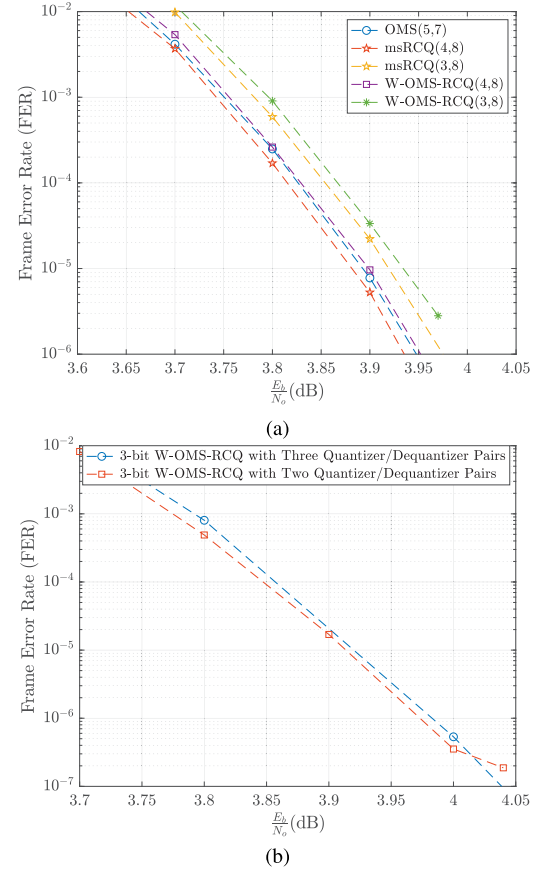


Fig. 9. Fig. (a): FER performance of W-Oms-RCQ decoders, RCQ decoders, and 5-bit OMS decoder for a (9472, 8192) QC LDPC code. Fig. (b): FER performance of 3-bit W-Oms-RCQ decoders with two and three quantizer/dequantizer pairs.

0.025 and 0.05 dB gap, respectively. Specifically, the 3-bit msRCQ decoder has similar LUT, BRAM, and routed net usage to the 4-bit W-Oms-RCQ decoder. On the other hand, the 3-bit W-Oms-RCQ uses much fewer resources than the 4-bit W-Oms-RCQ decoder.

The 3-bit W-Oms-RCQ decoder in Fig. 9a uses three quantizers for three decoding phases. In the first three iterations, most messages have low magnitudes. Hence a quantizer with small C is required for a finer resolution to the low-magnitude values. However, the message magnitudes increase with the increase of decoding iteration. As a result, the quantizers with larger C should be used correspondingly. Fewer quantizers may not accommodate the message magnitude growth in the decoding process and will result in performance degradation.

TABLE III
HARDWARE USAGE OF VARIOUS DECODING STRUCTURE FOR (9472,8192) QC-LDPC CODE

Decoding Structure	LUTs	Registers	BRAMS	Routed Nets
OMS(5,7) (baseline)	21127	12966	17	29202
msRCQ(4,8)	20355(↓ 3.6%)	13967(↑ 7.0%)	17.5(↑ .03%)	28916(↓ 1%)
msRCQ(3,8)	17865(↓ 15.4%)	12098(↓ 6.7%)	17(—)	25332(↓ 13.3%)
W-OMS-RCQ(4,8)	17645(↓ 16.5%)	13297(↑ 2.6%)	17(—)	25361(↓ 13.2%)
W-OMS-RCQ(3,8)	16306(↓ 22.82%)	12104(↓ 6.65%)	17(—)	23252(↓ 20.38%)

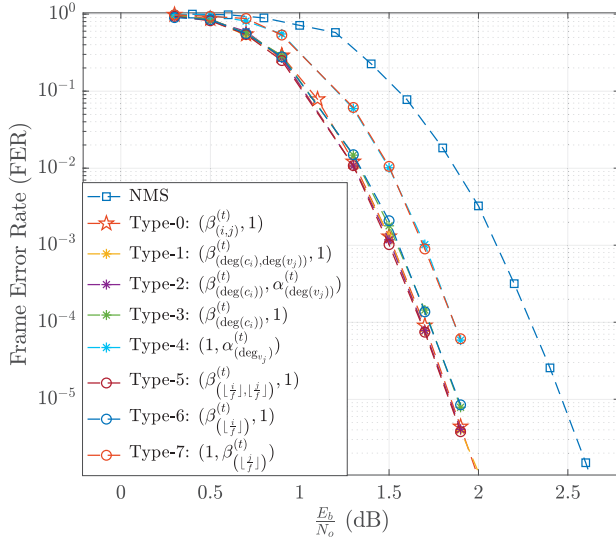


Fig. 10. FER performance of N-2D-NMS decoders with various weight sharing types for a (3096,1032) PBRL LDPC code compared with N-NMS (type 0) and NMS.

For example, Fig. 9b considers a 3-bit W-OMS-RCQ decoder that uses two quantizer/dequantizer pairs, the first pair has $C_1 = 3$, $\gamma_1 = 1.3$ and is used for iteration 1 ~ 7, the second pair has $C_2 = 5$, $\gamma_2 = 1.3$ and is used for iteration 8 ~ 10. The simulation result shows that the 3-bit W-OMS-RCQ decoder that uses 2 quantizer/dequantizer pairs has an early error floor at FER of 10^{-7} .

C. $k = 1032$ Protograph-Based Raptor-Like Code

5G LDPC codes have the protograph-based raptor-like (PBRL) [35] structure which offers inherent rate-compatibility and excellent decoding performance. In this subsection, we examine the performance of N-2D-NMS decoders and W-RCQ decoders for a $k = 1032$ PBRL LDPC code, whose supported rates are listed in Table I. The edge distribution of the lowest-rate code, which corresponds to the full parity check matrix, is also given in Table I. All the decoders in this subsection are layer-scheduled with a maximum of 10 decoding iterations.

Fig. 10 shows the FER performance of the N-2D-NMS decoder with various weight sharing types for the PBRL code with lowest code rates $\frac{1}{3}$. As a comparison, the decoding performance of the N-NMS (type 0) decoder and the NMS decoder is also given. All of the decoders use floating-point message representation. The simulation results show that the N-NMS decoder has a more than 0.5 dB improvement over the NMS decoder but requires 1.6×10^4 parameters per iteration, as given in Table I. On the other hand, the N-2D-NMS decoders with types 1, 2, and 5 have the same decoding

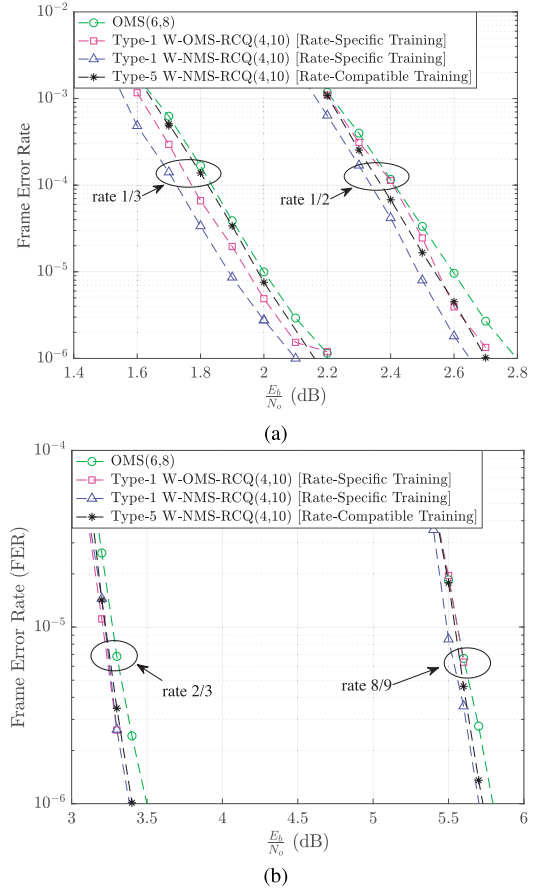


Fig. 11. FER performance of 4-bit W-RCQ decoders for $k = 1032$ PBRL code with different code rates. The term “rate-specific” means designing distinct decoders for each code rate; “rate-compatible” means training one decoder that matches all code rates. The 6-bit OMS decoder is given as a comparison.

performance as the N-NMS decoder but only use 41, 15, and 101 parameters in each iteration, respectively. By only considering sharing weights based on check node degrees, N-2D-NMS decoders of types 3 and 6 have a degradation of around 0.05 dB compared with the N-NMS decoder, with 8 and 17 parameters in each iteration, respectively. On the other hand, when only considering sharing the weights based on the variable node degrees, N-2D-NMS decoders of types 4 and 7 have a degradation of around 0.2 dB compared with the N-NMS decoder, with 7 and 25 parameters in each iteration, respectively. Thus, for this (3096,1032) PBRL LDPC code, assigning weights based only on check nodes can benefit more than assigning weights based on variable nodes.

Fig. 11 gives the FER performance of fixed-point W-NMS-RCQ decoders for the $k = 1032$ PBRL code with rate $\frac{1}{3}$, $\frac{1}{2}$, $\frac{2}{3}$ and $\frac{8}{9}$. The W-NMS-RCQ decoder assigns 4 bits to C2V message and 10 bits to V2C message. Two

quantizer/dequantizer pairs are used for W-NMS-RCQ decoder across all investigated rates. The first quantizer has $C_1 = 7$, $\gamma_2 = 1.7$ and is used for the first 7 iterations. The second quantizer has $C_2 = 10$, $\gamma_2 = 2.3$ and is used for the last three iterations. We use a 6-bit OMS decoder as the benchmark because it delivers better decoding performance than the NMS decoder with the same bit width.

We first consider the 4-bit W-NMS-RCQ decoder with type-1 weight sharing that assigns the same weight to the edges with the same check node degree and variable node degree. The decoder is rate-specific; i.e., a W-NMS-RCQ decoder is trained separately for each considered code rate. The simulation results show that, targeting an FER of 10^{-6} , the 4-bit rate-specific W-NMS-RCQ decoder outperforms the 6-bit OMS decoder with $0.1 \sim 0.15$ dB for all considered code rates. Fig. 11 also gives the FER curves of the 4-bit type-1 rate-specific W-OMS-RCQ decoder at various code rates. The simulation indicates that W-OMS-RCQ doesn't perform as well as the W-NMS-RCQ decoder.

For the PBRL code, the protomatrix of each possible rate is a sub-matrix of a base protomatrix [35]. As shown in Table. I, the type-5 weight sharing assigns the same weight to the edges corresponding to the same element in the protomatrix. Hence, it is possible to use *one* trained type-5 neural decoder to match different code rates. We refer to such a decoder as a rate-compatible decoder. In [29], the authors propose training the rate-compatible decoder using samples from different code rates.

Fig. 11 shows the decoding performance of the rate-compatible type-5 W-NMS-RCQ decoder. The simulation result shows that for the higher rate, such as $\frac{2}{3}$ and $\frac{8}{9}$, the rate-compatible type-5 W-NMS-RCQ decoder has a similar decoding performance to the rate-specific type-1 W-NMS-RCQ decoder. However, for the lower rates such as $\frac{1}{3}$ and $\frac{1}{2}$, the rate-compatible type-5 W-NMS-RCQ decoder method doesn't deliver decoding performance as well as rate-specific type-1 W-NMS-RCQ decoder. Besides, considering the four rates in Fig. 11, the number of neural weights for rate-specific type-1 and rate-compatible type-5 W-NMS-RCQ decoder are 96 and 101, respectively.

VI. CONCLUSION

Neural networks have improved MinSum message-passing decoders for low-density parity-check (LDPC) codes by multiplying or adding weights to the messages, where a neural network determines the weights. However, the neural network complexity to determine distinct weights for each edge is high, often limiting the application to relatively short LDPC codes. In particular, when training the neural network using PyTorch or TensorFlow, memory constraints prevent designing weights for long-blocklength codes. This paper solves this memory issue by compactly storing feed-forward messages, which allows us to design weights for blocklengths of 16,000 bits. As an additional contribution, this paper identifies a gradient explosion problem in the neural decoder training and provides a posterior joint training method that addresses this problem.

For neural decoders such as N-NMS decoder and N-OMS decoder, assigning distinct weights to each edge in each

decoding iteration is impractical for long-blocklength codes because of the storage burden associated with the huge number of the neural weights. This paper proposes node-degree-based weight-sharing schemes that drastically reduce the number of required weights with often negligible increase in frame error rate.

Finally, this paper combines the idea of weights designed by a neural network with the nonlinear quantization paradigm of RCQ, producing the W-RCQ decoder, a non-uniformly quantized decoder that delivers excellent decoding performance in the low-bitwidth regime. Unlike the RCQ decoder, which designs quantizer/dequantizer pairs for each layer and iteration, the W-RCQ decoder only uses a small number of quantizer/dequantizer pairs.

ACKNOWLEDGMENT

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect views of NSF.

REFERENCES

- [1] L. Wang, S. Chen, J. Nguyen, D. Dariush, and R. Wesel, "Neural-network-optimized degree-specific weights for LDPC MinSum decoding," in *Proc. 11th Int. Symp. Topics Coding (ISTC)*, Aug. 2021, pp. 1–5.
- [2] R. G. Gallager, "Low-density parity-check codes," *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, Jan. 1962.
- [3] E. Nachmani, Y. Be'ery, and D. Burshtein, "Learning to decode linear codes using deep learning," in *Proc. 54th Annu. Allerton Conf. Commun., Control, Comput. (Allerton)*, Sep. 2016, pp. 341–346.
- [4] L. Lugosch and W. J. Gross, "Neural offset min-sum decoding," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jun. 2017, pp. 1361–1365.
- [5] E. Nachmani, E. Marciano, D. Burshtein, and Y. Be'ery, "RNN decoding of linear block codes," 2017, *arXiv:1702.07560*.
- [6] E. Nachmani, E. Marciano, L. Lugosch, W. J. Gross, D. Burshtein, and Y. Be'ery, "Deep learning methods for improved decoding of linear codes," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 119–131, Feb. 2018.
- [7] F. Liang, C. Shen, and F. Wu, "An iterative BP-CNN architecture for channel decoding," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 144–159, Feb. 2018.
- [8] X. Wu, M. Jiang, and C. Zhao, "Decoding optimization for 5G LDPC codes by machine learning," *IEEE Access*, vol. 6, pp. 50179–50186, 2018.
- [9] L. Lugosch and W. J. Gross, "Learning from the syndrome," in *Proc. 52nd Asilomar Conf. Signals, Syst., Comput.*, Oct. 2018, pp. 594–598.
- [10] W. Lyu, Z. Zhang, C. Jiao, K. Qin, and H. Zhang, "Performance evaluation of channel decoding with deep neural networks," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2018, pp. 1–6.
- [11] X. Xiao, B. Vasic, R. Tandon, and S. Lin, "Finite alphabet iterative decoding of LDPC codes with coarsely quantized neural networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2019, pp. 1–6.
- [12] C. Deng and S. L. Bo Yuan, "Reduced-complexity deep neural network-aided channel code decoder: A case study for BCH decoder," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2019, pp. 1468–1472.
- [13] A. Abotabl, J. H. Bae, and K.-B. Song, "Offset min-sum optimization for general decoding scheduling: A deep learning approach," in *Proc. IEEE 90th Veh. Technol. Conf. (VTC-Fall)*, Sep. 2019, pp. 1–5.
- [14] A. Buchberger, C. Häger, H. D. Pfister, L. Schmalen, and A. Graell i Amat, "Pruning and quantizing neural belief propagation decoders," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 1957–1966, Jul. 2021.
- [15] Q. Wang, S. Wang, H. Fang, L. Chen, L. Chen, and Y. Guo, "A model-driven deep learning method for normalized min-sum LDPC decoding," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, Jun. 2020, pp. 1–6.
- [16] M. Lian, F. Carpi, C. Häger, and H. D. Pfister, "Learned belief-propagation decoding with simple scaling and SNR adaptation," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jul. 2019, pp. 161–165.

- [17] J. Nguyen et al., "Neural normalized min-sum message-passing vs. Viterbi decoding for the CCSDS line product code," 2021, *arXiv:2111.07959*.
- [18] S. K. Planjery, D. Declercq, L. Danjean, and B. Vasic, "Finite alphabet iterative decoders—Part I: Decoding beyond belief propagation on the binary symmetric channel," *IEEE Trans. Commun.*, vol. 61, no. 10, pp. 4033–4045, Oct. 2013.
- [19] X. Xiao, B. Vasic, R. Tandon, and S. Lin, "Designing finite alphabet iterative decoders of LDPC codes via recurrent quantized neural networks," *IEEE Trans. Commun.*, vol. 68, no. 7, pp. 3963–3974, Jul. 2020.
- [20] J. Lewandowsky and G. Bauch, "Information-optimum LDPC decoders based on the information bottleneck method," *IEEE Access*, vol. 6, pp. 4054–4071, 2018.
- [21] M. Stark, J. Lewandowsky, and G. Bauch, "Information-optimum LDPC decoders with message alignment for irregular codes," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2018, pp. 1–6.
- [22] M. Stark, G. Bauch, L. Wang, and R. D. Wesel, "Information bottleneck decoding of rate-compatible 5G-LDPC codes," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–6.
- [23] J. K.-S. Lee and J. Thorpe, "Memory-efficient decoding of LDPC codes," in *Proc. Int. Symp. Inf. Theory, ISIT*, 2005, pp. 459–463.
- [24] X. He, K. Cai, and Z. Mei, "On mutual information-maximizing quantized belief propagation decoding of LDPC codes," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2019, pp. 1–6.
- [25] L. Wang, R. D. Wesel, M. Stark, and G. Bauch, "A reconstruction-computation-quantization (RCQ) approach to node operations in LDPC decoding," in *Proc. IEEE Global Commun. Conf.*, Dec. 2020, pp. 1–6.
- [26] C. Terrill et al., "FPGA implementations of layered MinSum LDPC decoders using RCQ message passing," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2021, pp. 1–6.
- [27] L. Wang et al., "Reconstruction-computation-quantization (RCQ): A paradigm for low bit width LDPC decoding," *IEEE Trans. Commun.*, vol. 70, no. 4, pp. 2213–2226, Apr. 2022.
- [28] I. J. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: <http://www.deeplearningbook.org>
- [29] J. Dai et al., "Learning to decode protograph LDPC codes," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 1983–1999, Jul. 2021.
- [30] L. Xie et al., "Weight-sharing neural architecture search: A battle to shrink the optimization gap," *ACM Comput. Surv.*, vol. 54, no. 9, pp. 1–37, Dec. 2022.
- [31] J. Zhang, M. Fossorier, D. Gu, and J. Zhang, "Improved min-sum decoding of LDPC codes using 2-dimensional normalization," in *Proc. IEEE Global Telecommun. Conf.*, Feb. 2005, pp. 1187–1192.
- [32] J. Zhang, M. Fossorier, D. Gu, and J. Zhang, "Two-dimensional correction for min-sum decoding of irregular LDPC codes," *IEEE Commun. Lett.*, vol. 10, no. 3, pp. 180–182, Mar. 2006.
- [33] *Digital Video Broadcasting (DVB); Second Generation Framing Structure, Channel Coding and Modulation Systems for Broadcasting, Interactive Services, News Gathering and Other Broadband Satellite Applications (DVB-S2)*, ETSI EN 302 307, 2019.
- [34] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.
- [35] T.-Y. Chen, K. Vakilinia, D. Divsalar, and R. D. Wesel, "Protograph-based raptor-like LDPC codes," *IEEE Trans. Commun.*, vol. 63, no. 5, pp. 1522–1532, May 2015.



Linfang Wang received the M.Sc. degree in information and communication engineering from the Harbin Institute of Technology, Harbin, China, in 2018. He is currently pursuing the Ph.D. degree with the University of California at Los Angeles, Los Angeles. His current research interest lies in coding theory, especially on LDPC code, trellis code, and the application of deep learning/machine learning on channel coding.



Caleb Terrill received the bachelor's degree in computer engineering from UCLA in 2022. He is currently pursuing the master's degree in electrical engineering with Stanford. His technical interests include computer architecture, FPGA design, and formal verification.



Dariush Divsalar (Life Fellow, IEEE) received the Ph.D. degree in electrical engineering from the University of California at Los Angeles (UCLA), in 1978. Since then, he has been with the Jet Propulsion Laboratory (JPL), California Institute of Technology (Caltech), where he has been a JPL Senior Research Scientist (since 1999) and a JPL Fellow (since 2015). At JPL, he has involved with developing state-of-the-art technology for advanced deep-space communications systems and future NASA space exploration. He is also an Adjunct

Professor with the Electrical and Computer Engineering Department, UCLA. In 1986, he has taught graduate courses in communications and coding with UCLA and Caltech. He has published more than 280 technical papers, coauthored a book entitled *An Introduction to Trellis Coded Modulation with Applications*, contributed to three other books, and holds 28 U.S. patents. He was a co-recipient of the 1988 Best Paper Award of IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY. He was also a co-recipient of the joint Best Paper Award of the IEEE Information Theory and IEEE Communication Theory societies in 2008. The IEEE Communication Society has selected one of his papers for inclusion in a book entitled *The Best of the Best: Fifty Years of Communications and Networking Research*, containing the best 56 key research papers ever published in the society's 50-year history. He has received over 50 NASA Tech Brief Awards and the NASA Exceptional Engineering Achievement Medal in 1996. He was the 2014 IEEE Alexander Graham Bell Medal recipient. He received the Ellis Island Medal of Honor in 2023. He also received the NASA Distinguished Public Service Medal in 2023. He served as an Area Editor for IEEE TRANSACTIONS ON COMMUNICATIONS, from 1989 to 1996.



Richard D. Wesel (Fellow, IEEE) received the B.S. and M.S. degrees in electrical engineering from the Massachusetts Institute of Technology in 1989, and the Ph.D. degree in electrical engineering from Stanford University in 1996. He is a Professor with the Electrical and Computer Engineering Department, UCLA, where he is also the Associate Dean for Academic and Student Affairs with the Henry Samueli School of Engineering and Applied Science. His research is in the area of communication theory, with particular interest in low-density parity-check coding, short-blocklength communication with feedback, and coding for storage. He has received the National Science Foundation CAREER Award, the Okawa Foundation Award for research in information theory and telecommunications, and the Excellence in Teaching Award from the Samueli School of Engineering. He has served as an Associate Editor for Coding and Coded Modulation for IEEE TRANSACTIONS ON COMMUNICATIONS.